

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics

Practical – 11

Roll No.: T050	Name: Aashu Yadav
Class: TYIT	Batch: 1
Date of Assignment: 03/10/26	Date/Time of Submission: 03/10/26

Part A: Optimization Modeling

Business Problem: A company produces two products, Product A and Product B. The company wants to maximize its profit using the available machines and labour hours.

Given data:

Product Profit per Unit Machine Hours Labour Hours ;
A 40 2 1 ; B 30 1 2 ; Available Machine Hours = 100 ;
Available Labour Hours = 80

Practical Questions:

1. Define the decision variables for Product A and Product B.

Code:

```
#Import Library
from scipy.optimize import linprog
#Define Decision Variables
# x = units of Product A
# y = units of Product B
```

2. Define the objective of the problem.

Code:

```
x = units of Product A
y = units of Product B
```

3. Formulate the objective function to maximize the total profit.

Code:

```
c = [-40, -30]
```

4. Formulate the machine-hour constraint.

Code:

```
machine = [2, 1]
```

Output:

5. Formulate the labour-hour constraint.

Code:

```
labour = [1, 2]
```

6. Add non-negative constraints for the decision variables.

Code:

```
bounds = [(0, None), (0, None)]
```

7. Solve the linear programming problem using Python.

Code:

```
result = linprog(
    c,
    A_ub=[
        [2, 1],
```

```
        [1, 2]
    ],
    b_ub=[100, 80],
    bounds=bounds,
    method="highs"
)
```

print(result)

Output:

```
message: Optimization terminated successfully.
success: True
status: 0
      fun: -2200.0
         x: [ 4.000e+01  2.000e+01]
        nit: 2
  lower: residual: [ 4.000e+01  2.000e+01]
        marginals: [ 0.000e+00  0.000e+00]
  upper: residual: [          inf          inf]
        marginals: [ 0.000e+00  0.000e+00]
    eqlin: residual: []
          marginals: []
  ineqlin: residual: [ 0.000e+00  0.000e+00]
          marginals: [-1.667e+01 -6.667e+00]
 mip_node_count: 0
 mip_dual_bound: 0.0
      mip_gap: 0.0
```

8. Find the optimal number of units of Product A and Product B.

Code:

```
A = result.x[0]
B = result.x[1]
print("Product A:", A)
print("Product B:", B)
```

Output:

```
Product A: 40.0
Product B: 20.0
```

9. Calculate the maximum profit.

Code:

```
profit = 40 * A + 30 * B
print("Maximum Profit:", profit)
```

Output:

```
Maximum Profit: 2200.0
```

10. Check whether the available machine and labour hours are satisfied.

Code:

```
machine_used = 2 * A + B
labour_used = A + 2 * B
print("Machine Hours Used:", machine_used)
```

Aashu Yadav
T050

```
print("Labour Hours Used:", labour_used)
```

Output:

```
Machine Hours Used: 100.0
Labour Hours Used: 80.0
```

11. Display the optimal solution in a suitable format.

Code:

```
print("\nOptimal Solution")
print("-----")
print("Product A:", A, "units")
print("Product B:", B, "units")
print("Maximum Profit: ₹", profit)
```

12. Interpret the optimal solution in a business context.

- The company should produce 40 units of Product A and 20 units of Product B.
- This production plan uses the available machine and labour hours and gives a maximum profit of ₹2200.

13. Write a conclusion based on the optimization result.

The optimization model determines the best production plan. The company should produce 40 units of Product A and 20 units of Product B, giving a maximum profit of ₹2200 while satisfying the available machine and labour hour constraints.

Part B: Simulation Modeling

Business Problem: A supermarket wants to decide the appropriate inventory level for a product when daily demand is uncertain.

Practical Questions:

14. Generate random daily demand values for a product for 30 days.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)
demand = np.random.randint(10, 51, 30)
print("Daily Demand:")
print(demand)
```

Output:

```
Daily Demand:
[48 38 24 17 30 48 28 32 20 20 33 45 49 33 12 31 11 33 39 47
 31 34 36 37 25 24]
```

15. Display the generated demand values.

Code:

```
print("Demand for 30 Days:")
for day, d in enumerate(demand, 1):
    print("Day", day, ":", d, "units")
```

Output:

```
Demand for 30 Days:
Day 1 : 48 units
Day 2 : 38 units
Day 3 : 24 units
Day 4 : 17 units
Day 5 : 30 units
Day 6 : 48 units
Day 7 : 28 units
Day 8 : 32 units
Day 9 : 20 units
Day 10 : 20 units
```

16. Select an inventory level of 30 units.

Code:

```
print("Machine Hours Used:", machine_used)
```

```
print("Labour Hours Used:", labour_used)
```

Output:

```
Optimal Solution
-----
Product A: 40.0 units
Product B: 20.0 units
Maximum Profit: ₹ 2200.0
Machine Hours Used: 100.0
Labour Hours Used: 80.0
```

```
inventory = 30
```

```
print("Inventory Level:", inventory)
```

Output:

```
Inventory Level: 30
```

17. Compare the daily demand with the available inventory.

Code:

```
for day, d in enumerate(demand, 1):
    if d <= inventory:
        print("Day", day, ": Demand =", d, "Inventory =", inventory)
    else:
        print("Day", day, ": Demand =", d, "Inventory =", inventory,
              "Shortage")
```

Output:

```
Day 1 : Demand = 48 Inventory = 30 Shortage
Day 2 : Demand = 38 Inventory = 30 Shortage
Day 3 : Demand = 24 Inventory = 30
Day 4 : Demand = 17 Inventory = 30
Day 5 : Demand = 30 Inventory = 30
Day 6 : Demand = 48 Inventory = 30 Shortage
Day 7 : Demand = 28 Inventory = 30
Day 8 : Demand = 32 Inventory = 30 Shortage
Day 9 : Demand = 20 Inventory = 30
Day 10 : Demand = 20 Inventory = 30
```

18. Calculate the number of units sold each day.

Code:

```
units_sold = np.minimum(demand, inventory)
```

```
print("Units Sold Each Day:")
```

```
print(units_sold)
```

Output:

```
Units Sold Each Day:
[30 30 24 17 30 30 28 30 20 20 30 30 30 30 12 30 11 30 30 30
 30 30 30 30 25 24]
```

19. Calculate the number of units left after each day.

Code:

```
shortage = np.maximum(demand - inventory, 0)
```

```
print("Shortage Each Day:")
```

```
print(shortage)
```

Output:

```
Shortage Each Day:
[18  8  0  0  0 18  0  2  0  0  3 15 19  3  0  1  0  3  9 17
 1  4  6  7  0  0]
```

20. Calculate the shortage when demand is greater than available inventory.

Code:

```
total_sold = units_sold.sum()
total_shortage = shortage.sum()
print("Total Units Sold:", total_sold)
print("Total Shortage:", total_shortage)
```

Output:

```
Total Units Sold: 783
Total Shortage: 146
```

21. Calculate the total units sold and total shortage for 30 days.

Code:

Output:

22. Repeat the simulation using inventory levels of 30, 40, and 50 units.

Code:

```
inventory_levels = [30, 40, 50]
results = []
for inventory in inventory_levels:
    units_sold = np.minimum(demand, inventory)
    shortage = np.maximum(demand - inventory, 0)
    total_sold = units_sold.sum()
    total_shortage = shortage.sum()
    results.append([
        inventory,
        total_sold,
        total_shortage
    ])
print(results)
```

Output:

```
[[30, np.int64(783), np.int64(146)], [40, np.int64(890),
```

23. Store the results of all three scenarios in a DataFrame.

Code:

```
df = pd.DataFrame(
    results,
    columns=["Inventory Level", "Total Sales", "Total Shortage"]
)
```

print(df)

Output:

26. Identify how changing the inventory level affects sales and shortage.

- As inventory level increases, total sales increase or remain stable.
- As inventory level increases, total shortage decreases.

27. Write a conclusion about the inventory level based on the simulation results.

The simulation shows that increasing the inventory level increases total sales and reduces. A suitable inventory level helps the supermarket meet customer demand while minimizing stock shortages.

	Inventory Level	Total Sales	Total Shortage
0	30	783	146
1	40	890	39
2	50	929	0

24. Compare the total sales and total shortage for each inventory level.

Code:

```
print("Comparison of Inventory Levels:")
```

```
print(df)
```

Output:

```
Comparison of Inventory Levels:
Inventory Level  Total Sales  Total Shortage
0               30         783             146
1               40         890              39
2               50         929              0
```

25. Create a chart to compare the results of the three scenarios.

Code:

```
plt.bar(df["Inventory Level"] - 2, df["Total Sales"],
        width=4, label="Total Sales")
plt.bar(df["Inventory Level"] + 2, df["Total Shortage"],
        width=4, label="Total Shortage")
plt.xlabel("Inventory Level")
plt.ylabel("Units")
plt.title("Inventory Level vs Sales and Shortage")
plt.legend()
plt.show()
```

Output:

